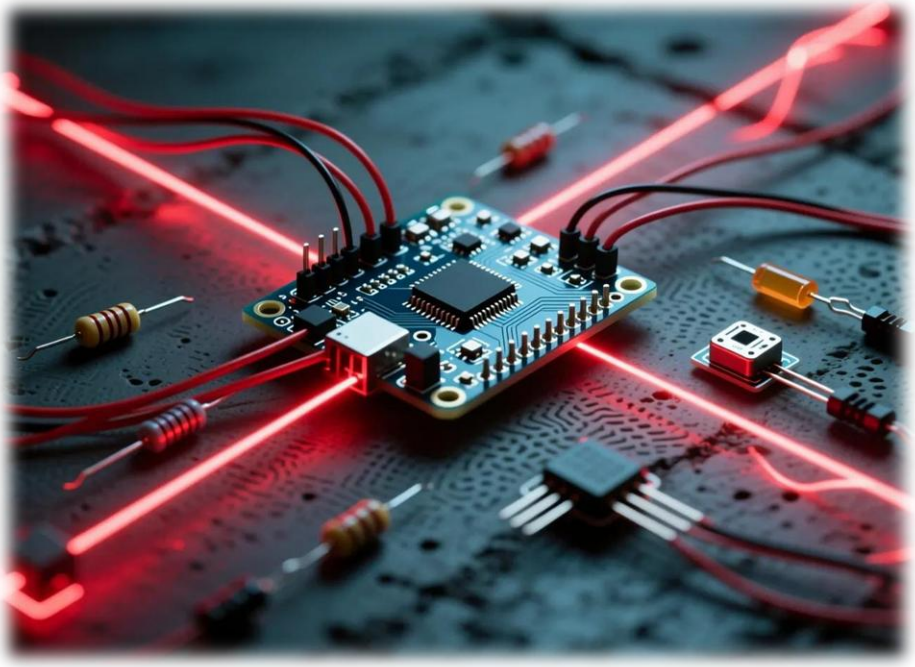
Rapport du projet :

Système de Sécurité Intelligent Basé sur Détection de Mouvement et IA



SOMMAIRE

➔ **Introduction générale**

➔ **Présentation du projet**

➔ **Composantes utilisées**

➔ **Schéma et réalisation sur Proteus**

➔ **Fonctionnement général du système**

➔ **Code Arduino utilisé dans la simulation**

➔ **Code Python (capture d'image et envoi à l'IA)**

➔ **Réalisation réel**

➔ **Limites du système**

➔ **Conclusion générale**

Introduction générale :

Dans un contexte où la sécurité devient un besoin essentiel, il est possible de concevoir des systèmes intelligents efficaces et à coût réduit grâce à l'électronique embarquée et à l'Intelligence Artificielle.
Ce projet consiste à réaliser un **système de sécurité intelligent** basé sur la détection de mouvement et l'analyse par IA.

Lorsqu'un mouvement est détecté par un capteur PIR, l'Arduino envoie une alerte au PC, qui active alors la webcam pour prendre une photo de la personne. Cette photo peut ensuite être analysée par une IA pour déterminer si la personne est autorisée ou non.

Le système est conçu pour être **simple, fonctionnel et très économique** , tout en restant évolutif .

Représentation du projet:

➔ le fonctionnement global :

1. **Le capteur PIR** détecte un mouvement humain.
2. **L'Arduino** reçoit l'information et allume :
   - la **LED rouge** si un mouvement est détecté,
   - la **LED verte** lorsque tout est normal.
3. L'Arduino envoie un message à l'ordinateur via le port série (COM 9).
4. L'ordinateur déclenche **la webcam intégrée** pour scanner le visage.
5. La biblio « face_recognition » analyse la personne si elle est connue ou pas.
6. Envoie de message + photo sur **le bot telegram** .

Composantes utilisées:

Le matériel utilisé est simple et peu coûteux :

- **Arduino Uno**

Carte microcontrôleur assurant la lecture du capteur et l'envoi de signaux.



- **Capteur PIR (infrarouge passif)**

Détecte la présence humaine grâce aux variations d'infrarouges.



- **LED rouge**

Indique l'état *alerte* (mouvement détecté).



- **LED verte**

Indique l'état *normal* (aucun mouvement).

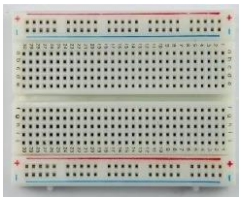


- **Deux résistances**

Pour protéger les LEDs.



- **Breadboard .**



- **Câbles de connexion**

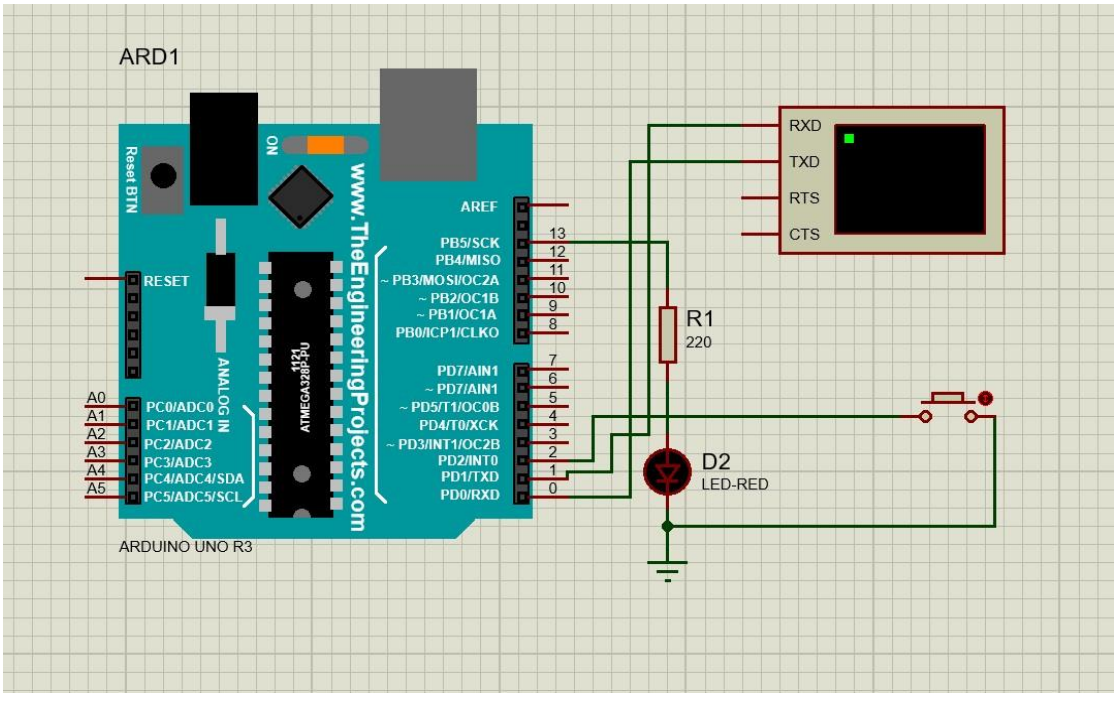


Pour relier tous les éléments.

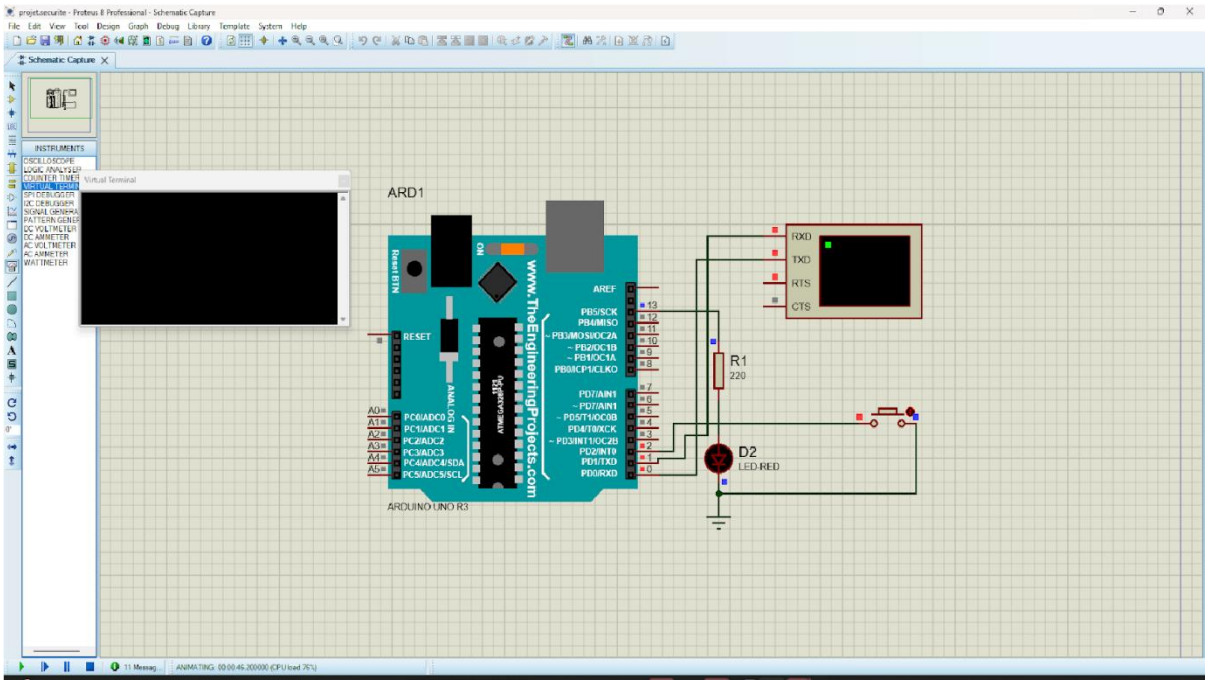
- **Webcam du PC**

Capture photo lors de la détection.

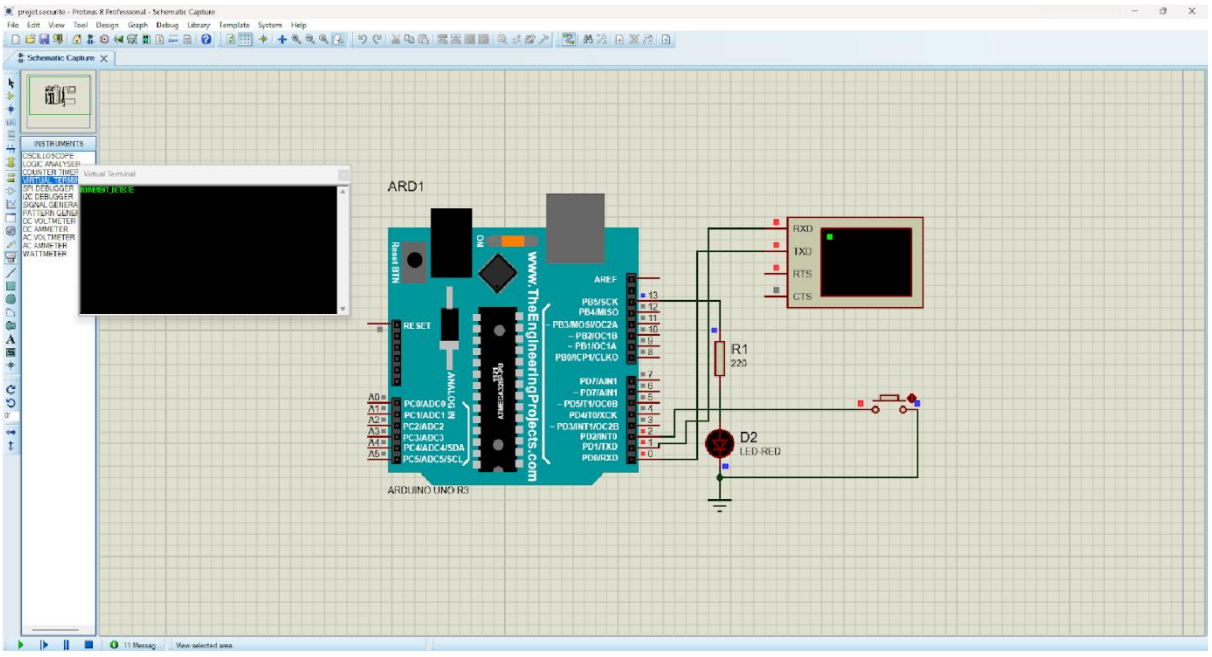
Schéma du montage (Proteus):



Avant détection:



Après détection :



Code Arduino:

```
//  Détection PIR / bouton poussoir
int pirPin = 2;

//  LEDs
int ledAutorisee = 12;  // LED verte
int ledNonAutorisee = 13; // LED rouge

void setup() {
  pinMode(pirPin, INPUT_PULLUP);

  pinMode(ledAutorisee, OUTPUT);
  pinMode(ledNonAutorisee, OUTPUT);

  Serial.begin(9600);
}

void loop() {
  int etat = digitalRead(pirPin);

  if (etat == LOW) {
    Serial.println("MOTION");
    delay(500);
  }

  //  Lecture du message Python
  if (Serial.available()) {
    String msg = Serial.readStringUntil('\n');

    if (msg == "OK") {
      digitalWrite(ledAutorisee, HIGH);
      digitalWrite(ledNonAutorisee, LOW);
    }
    else if (msg == "ALARM") {
      digitalWrite(ledNonAutorisee, HIGH);
      digitalWrite(ledAutorisee, LOW);
    }
  }
}
```

Code Python:

```
import cv2
import serial
import time
import face_recognition
import os
import numpy as np
from telegram import Bot

# Configuration Telegram
BOT_TOKEN = "ton BOT_TOKEN" # Le bot_token est une information personnel
CHAT_ID = "Ton chat ID " # Le chat ID est une information personnel
bot = Bot(token=BOT_TOKEN)

# Chargement des visages connus
known_faces = []
known_names = []

path = r"authorized"  # dossier contenant les photos des personnes autorisées
files = [f for f in os.listdir(path) if f.lower().endswith((".jpg", ".png"))]

for f in files:
    try:
        img = cv2.imread(os.path.join(path, f), cv2.IMREAD_UNCHANGED)

        if img is None:
            print(f"⚠ Impossible de lire {f}, image ignorée")
            continue

        if img.dtype != np.uint8:
            img = (255 * img).astype(np.uint8)

        if len(img.shape) == 2:
            img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)
        elif img.shape[2] == 4:
            img = cv2.cvtColor(img, cv2.COLOR_RGBA2RGB)
        elif img.shape[2] == 3:
            pass
        else:
            print(f"⚠ Format inconnu pour {f}, image ignorée")
            continue

        encodings = face_recognition.face_encodings(img)
```

```
        if len(encodings) > 0:
            known_faces.append(encodings[0])
            known_names.append(os.path.splitext(f)[0])
        else:
            print(f"⚠ Aucun visage détecté dans {f}")

    except Exception as e:
        print(f"⚠ Erreur avec {f} : {e}")

print(f"☑ {len(known_faces)} visage(s) connu(s) chargé(s).")

# Connexion Arduino (port COM9)
arduino_port = "COM9"   # <<< PORT FIXE
try:
    arduino = serial.Serial(arduino_port, 9600)
    time.sleep(2)
    print(f"☑ Arduino connecté sur {arduino_port}\n")
except Exception as e:
    raise Exception(f"⚠ Impossible d'ouvrir {arduino_port} : {e}")

# Initialisation de la caméra
cam = cv2.VideoCapture(0)
if not cam.isOpened():
    raise Exception("Impossible d'ouvrir la caméra !")

# Fonction de reconnaissance faciale
def recognize(frame):
    if frame is None:
        return "INCONNU"

    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    faces = face_recognition.face_locations(rgb)
    encodings = face_recognition.face_encodings(rgb, faces)

    for enc in encodings:
        matches = face_recognition.compare_faces(known_faces, enc)
        face_distances = face_recognition.face_distance(known_faces, enc)

        if True in matches:
            i = np.argmin(face_distances)
            return known_names[i]

    return "INCONNU"

# Boucle principale
try:
    while True:
        if arduino.in_waiting > 0:
            message = arduino.readline().decode().strip()
```

```
            if message == "MOTION":
                print("➡ Mouvement détecté !")

                ret, frame = cam.read()
                if not ret:
                    print("Erreur caméra.")
                    continue

                cv2.imwrite("capture.jpg", frame)

                # Reconnaissance faciale
                person = recognize(frame)

                if person != "INCONNU":
                    text = f"✓ Personne connue : {person}"
                    arduino.write(b"OK\n")      # LED autorisée
                else:
                    text = "⚠ Personne inconnue !!!"
                    arduino.write(b"ALARM\n")   # LED non autorisée

                # Envoi Telegram
                try:
                    with open("capture.jpg", "rb") as photo:
                        bot.sendPhoto(chat_id=CHAT_ID, photo=photo)
                    bot.sendMessage(chat_id=CHAT_ID, text=text)
                    print(f"📩 Telegram envoyé : {text}")

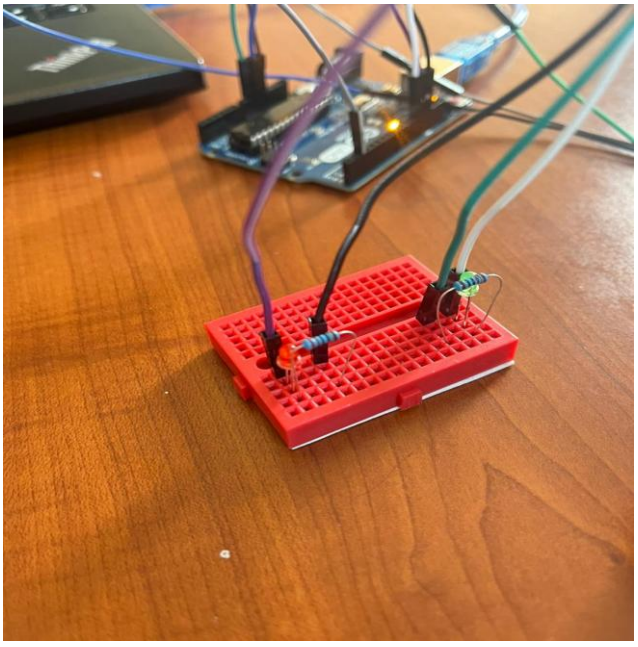
                except Exception as e:
                    print(f"Erreur Telegram : {e}")

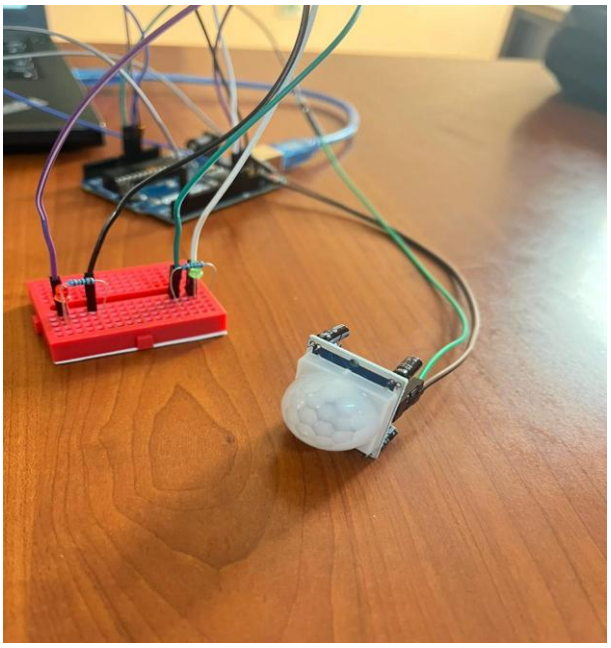
                time.sleep(3)

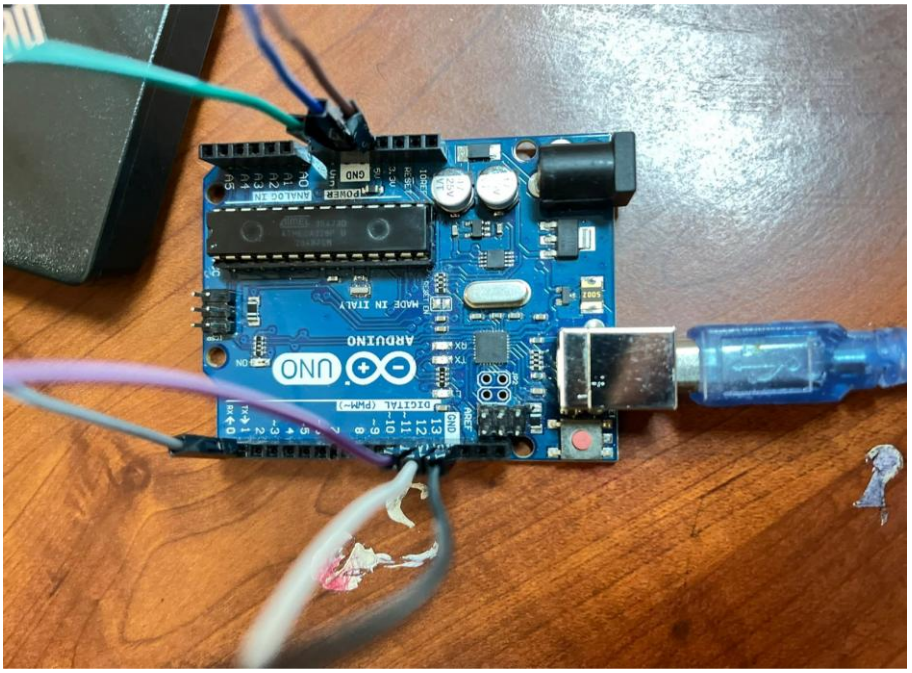
except KeyboardInterrupt:
    print("Arrêt manuel.")

finally:
    cam.release()
    arduino.close()
    print("Fermeture complète.")
```

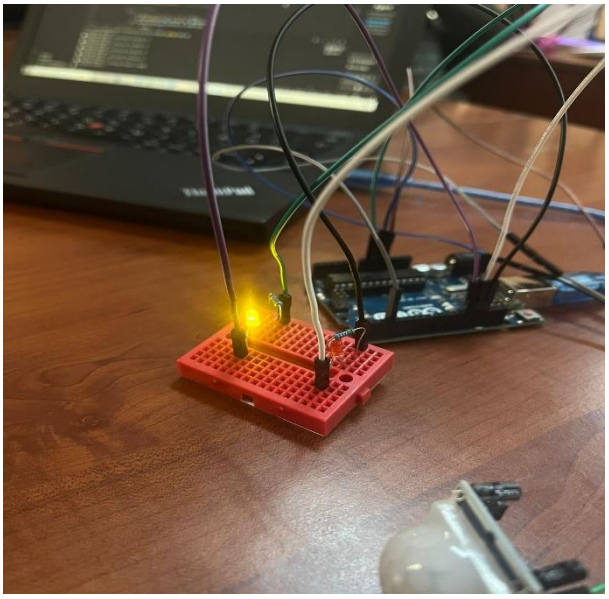
Réalisation réel :



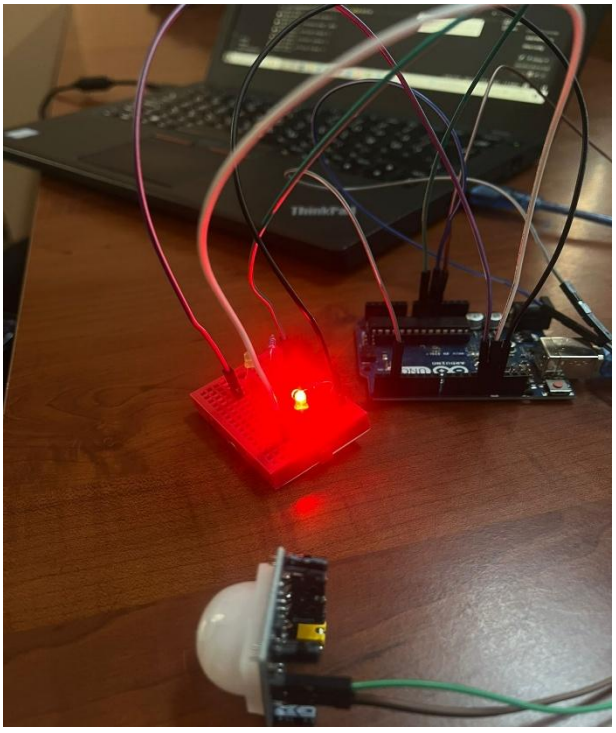




La LED verte s’allume quand la personne est connue :



La LED rouge s’allume quand la personne n’est pas connue :



Limites du projet

Malgré son efficacité et son très faible coût, le système présente plusieurs limites techniques liées à la fois à la simulation sous Proteus et à la réalisation réelle :

➔ **Simulation limitée dans Proteus**

- Le capteur PIR n'existe pas nativement dans Proteus, ce qui a obligé à simuler le mouvement avec un simple bouton.
  → Cela ne reproduit pas fidèlement le comportement réel du capteur (temps de stabilisation, bruit, sensibilité).

- La webcam ne peut pas être simulée dans Proteus.
  → Impossible de tester la capture d'image ou la reconnaissance faciale directement dans la simulation.

- Le fonctionnement du port série entre Proteus → Python n'est pas totalement réaliste.

➔ **Reconnaissance faciale basique**

- La reconnaissance faciale utilisée côté Python dépend de la qualité de la webcam du PC.

- Risque de fausses reconnaissances si la lumière est faible.

➔ **Dépendance à l'ordinateur**

- Le système nécessite :
  - un PC allumé,
  - une webcam active,
  - le script Python en cours d'exécution.

- Le système ne peut pas fonctionner autonome, uniquement avec Arduino.

➔ **Latence possible entre Arduino → Python**

- Petite latence au moment du transfert des messages via port série.

- Peut provoquer un léger retard entre détection du mouvement et prise de photo.

➔ **LEDs simples → pas de retour utilisateur avancé**

- Seule une LED rouge/verte indique l'état du système.

- Aucune interface visuelle ou sonore plus poussée.

Conclusion générale :

Ce projet de système de sécurité intelligent démontre qu'il est tout à fait possible de réaliser une solution efficace, fiable et à très faible coût en combinant des technologies simples et accessibles. Grâce à l'utilisation d'un capteur infrarouge de mouvement, d'une carte Arduino et d'une webcam, le système est capable de détecter automatiquement un mouvement, de capturer une image et même de vérifier l'identité d'une personne à l'aide d'un module d'intelligence artificielle.

La simulation sur Proteus Design Suite 8.17 a joué un rôle essentiel, car elle nous a permis de tester l'ensemble du fonctionnement avant de passer à la partie pratique. Cela garantit un montage plus propre, réduit les erreurs et facilite la compréhension du comportement du système.

Ce projet montre également qu'avec un peu de programmation et des composants simples, on peut développer des solutions proches de celles utilisées dans le domaine professionnel. En plus d'être économique, le système est flexible, évolutif et peut être amélioré facilement (ajout d'alarme, envoi d'alertes, stockage sur cloud, etc.).

En résumé, ce travail nous a permis de comprendre comment intégrer électronique, simulation et intelligence artificielle pour concevoir un système de sécurité moderne, accessible et réaliste. C'est un projet formateur qui ouvre la voie à d'autres applications plus avancées dans le domaine de la surveillance intelligente.